AI Agent Integration Plan for Fitness Tracker

This document outlines a practical, professional 10-point plan to integrate AI agent(s) with the Fitness Tracker app so agents can securely access user data, call external LLMs/tools, and produce reports, summaries, and recommendations. The plan aligns with modern agent patterns, security practices, and your Render deployment model.

1) Target outcomes and success criteria

- Generate personalized weekly summaries and recommendations using ~20 core user data points (scalable to 100s across JSON/structured data/vector DB).
- Agents can: (a) read user data via API, (b) call external LLMs and tools, (c) write back structured results (reports/recs) to the app.
- Latency: <5s for cached/short answers; <20s for agentic multi-tool runs (with streaming UI feedback).
- Reliability: >99% success for agent runs; graceful fallbacks when external APIs fail.
- Security: data isolation per user; no sensitive prompts/PII in logs; auditable runs.

2) Reference architecture (Render-first)

- Keep existing Express API as the core backend (Node 18+), add an `agent` module and routes.
- Persistence:
  - Postgres (already used) as system of record for user data, agent jobs, and outputs.
  - Optional: add a vector DB (pgvector extension) for semantic search over historical notes/reports.
- Agent runtime:
  - Option A (recommended): Server-side agent orchestrator inside the Web Service (Express). Simple to deploy; use background queue for longer jobs.
  - Option B: A separate Worker service on Render for long-running/queued agent jobs.
- External providers/tools:
  - LLMs (OpenAI, Azure OpenAI, Anthropic) via provider SDKs.
  - Retrieval tools: HTTP fetch, web search (if needed), domain APIs.
  - Safety tools: PII scrubbing and prompt-injection guard.

3) Data contract and schemas

- Define a minimal contract for agent input/output.
  - Input: user_id, time_window (e.g., 7d), goals (array), core metrics (weight, steps, workouts, calories), free-text notes.
  - Output: report_id, timeframe, key insights (array), recommendations (array with rationale/actions), citations (urls), confidence.
- Postgres tables (illustrative):
  - `agent_runs(id, user_id, status, started_at, finished_at, model, tools, error_text)`
  - `agent_outputs(id, run_id, user_id, title, timeframe, summary, insights_json, recommendations_json, citations_json, created_at)`
  - Optional `embeddings(doc_id, user_id, chunk, embedding vector)` for retrieval.

4) Secure agent API endpoints

- POST `/api/agent/run` (auth required): enqueue a run for a user/timeframe/options; returns run_id.
- GET `/api/agent/run/:run_id` (auth): status + partial or final output.
- GET `/api/agent/outputs` (auth): list outputs for user.
- GET `/api/agent/outputs/:id` (auth): fetch a specific report.
- Use JWT already in place; verify `req.user.sub` for access control. Rate limit per user.

5) Orchestration logic (agent core)

- Implement a modular agent pipeline:
    1. Load user context: profile, goals, entries (existing endpoints/queries) for the window.
    2. Optional retrieval: vector search over past reports/notes.
    3. Tool-augmented reasoning: call LLM with a system prompt that includes the contract and safety rules; allow tools:
        - `fetch(url)`: get trusted sources (PubMed/CDC) summaries.
        - `math/analyze`: small helper for numeric trends.
    4. Summarize and structure output per the schema.
    5. Persist output to `agent_outputs`; update `agent_runs`.
- Support streaming tokens to the client (Server-Sent Events or WebSocket) for responsiveness.

6) Prompting, grounding, and safety

- System prompt:
    - Role: evidence-based fitness coach; cite sources; avoid medical diagnosis; be actionable.
    - Constraints: do not expose secrets; avoid hallucination; prefer trusted sources.
- Retrieval grounding: include user's recent metrics and any verified articles; ask the LLM to cite URLs.
- Safety layers: input scrubbing (no secrets), output validation (JSON schema), profanity/hate filters.

7) Provider/tool abstraction

- Create a provider interface: `LLMProvider.generate({model, messages, tools})` with adapters for OpenAI, Azure OpenAI, Anthropic.
- Tool registry with capabilities and timeouts; log tool calls with redaction.
- Config via env vars: `LLM_PROVIDER`, `OPENAI_API_KEY` etc.; disable tools that require keys not present.

8) Background processing and retries

- For >10s tasks, enqueue jobs (e.g., a simple Postgres queue table or BullMQ + Redis if you add Redis).
- Worker loop (in the same process or a separate Render Worker): pulls queued jobs, runs the agent, updates status.
- Retries with exponential backoff for transient provider errors; circuit-break on repeated failures.

9) Observability and evaluation

- Structured logs for: run_id, user_id, model, tokens, tools, duration, errors (with redaction).
- Traces: add request ids, log timing of each phase.
- Evaluate with small golden datasets: verify insights correctness, actionability, and tone.
- Add `/api/agent/health` to surface provider/keys readiness.

10) Deployment and rollout on Render

- Environment vars in Web Service:
  - `DATABASE_URL` (already present), `LLM_PROVIDER`, `OPENAI_API_KEY` (or vendor key), `AGENT_MAX_TOKENS`, `AGENT_TIMEOUT_MS`.
- (Optional) Add a Worker service for queue processing; point to same repo and branch; share env group.
- Secure secrets via Render environment; never commit keys.
- Rollout plan:
  - Start with a single model (e.g., gpt-4o-mini) and daily summary tool; gather feedback.
  - Add weekly report with citations; then personalized recommendations with rationale.
  - Gate advanced tools behind feature flags.

Minimal implementation tasks (suggested)

1. DB migrations: add `agent_runs` and `agent_outputs` tables.
2. Add `POST /api/agent/run` and `GET /api/agent/run/:id` endpoints (auth required).
3. Create `agent/orchestrator.js` with provider abstraction and tool registry.
4. Add `LLMProvider` adapter (OpenAI or Azure) with streaming support.
5. Persist outputs; display in a new UI tab “Coach Reports”.
6. Add metrics logging and `/api/agent/health`.
7. Add simple queue (PG table) if jobs exceed ~10s; add Worker later if needed.

Security checklist

- Enforce per-user access; never allow cross-user data exposure.
- Redact PII/secrets in logs and tool call payloads.
- Enforce JSON schema for agent outputs; reject malformed results.
- Set strict timeouts and token limits; validate URLs before fetching.

Appendix: Example agent output JSON

```
{
  "report_id": "uuid",
  "timeframe": "2025-09-28..2025-10-04",
  "title": "Weekly Fitness Summary",
  "summary": "You averaged 8,500 steps/day and lost 0.6 lb...",
  "insights": [
    { "metric": "steps", "trend": "+12%", "note": "Above 8k on 5 days" },
    { "metric": "sleep", "trend": "-8%", "note": "Late bedtimes Fri/Sat" }
  ],
  "recommendations": [
    { "action": "Add 2x 20-min Zone 2 sessions", "rationale": "Build aerobic base;
aligns with goal fat loss" },
    { "action": "Lights out by 11pm on weekdays", "rationale": "Recover better;
target 7h+" }
  ],
  "citations": ["https://www.cdc.gov/physicalactivity/",
"https://pubmed.ncbi.nlm.nih.gov/"] ,
```

```
  "confidence": 0.84
}
```